GSTR-9 Scrutiny Register — file reference

GSTR-9 Scrutiny Register — every file, what it is and where it is

Each entry gives the path, what the file does, and why it does it that way. Nothing else — for how to run the system, see `run.md`.

Paths are relative to the checkout, written here as `<project>`. On the current deployment that is `~/Subbareddy/para-separator`, so `<project>/run.sh` means `~/Subbareddy/para-separator/run.sh`.

---

1. Files you run

`<project>/run.sh` — 136 lines

The whole pipeline, in order: inventory, text, OCR, notice split, table repair, reply extraction, workbook, checks.

Takes an optional dataset folder. That argument is resolved to an absolute path **before** the script changes directory, because it then moves into `pipeline/` and a relative path would stop resolving. From the folder name it derives three things: the workbook name, the workbook's destination (the folder itself), and a cache directory of its own under `work2/`, so two datasets never overwrite each other's text, OCR or replies.

Three things it does that look incidental and are not:

- **It derives `PATH` from the interpreter.** The stages shell out to `pdftotext`, `pdftoppm` and `tesseract`. Pointing `PY` at an interpreter whose environment holds those binaries, without putting its `bin` on `PATH`, once produced a silent full run in which every document extracted to zero characters and the empty results were cached. It now refuses to start if a binary is missing.
- **It relocates the scratch directory** to `work2/tmp` when the system one has under 2 GB free. OCR renders every page to PNG, which is gigabytes for a long scan; on a shared node `/tmp` was 100% full of other people's data and `pdftoppm` failed on every scanned PDF with nothing but a non-zero exit status.
- **It treats vision OCR as optional.** An endpoint that cannot accept images makes that stage warn and continue rather than killing the run, because it improves on tesseract's read — it is not a prerequisite for it.

`<project>/demo.sh` — 51 lines

One case, printed to the terminal instead of written to a workbook. Absolutises its arguments before changing directory, derives `PATH` the same way `run.sh` does, and relocates the scratch directory on the same rule.

`<project>/env.sh` — 46 lines

Keeps the interpreter and every cache inside the checkout, for a host where the home directory is off limits. Sets `CONDA_PKGS_DIRS`, `CONDA_ENVS_DIRS`, `MAMBA_ROOT_PREFIX`, `PIP_CACHE_DIR`, `XDG_CACHE_HOME`, `XDG_CONFIG_HOME`, `XDG_DATA_HOME`, `MPLCONFIGDIR`, `HF_HOME`, `TMPDIR`, `PY` and `PATH`, then prints what it resolved.

`CONDA_ENVS_PATH` is explicitly unset: micromamba aborts outright when both it and `CONDA_ENVS_DIRS` are set.

It only sets variables. Nothing is created until you run something.

---

2. The pipeline

`<project>/pipeline/paths.py` — 66 lines

Resolves the path variables — `GST_DATA`, `GST_WORK`, `GST_OUT`, `GST_OUT_DIR` — holds the role subfolder names (`DRC01_SCN`, `DRC 01 Reply`, `DRC 01 Order`), and provides `role_dirs()`.

`role_dirs()` matches those conventional names first and falls back, for any role they miss, to a loose match: *reply/response/submission*, *order/adjudicat*, *scn/notice/drc-01*. The order matters — every subfolder in this corpus is called "DRC 01 something", so the specific word decides and the generic DRC-01 pattern is tried last. Getting that wrong read the adjudication order as the notice.

`<project>/pipeline/params.py` — 185 lines

The 21 scrutiny parameters, from `Parameters_TN.pdf`: A1–A6 under-declaration of tax payable, B1–B10 excess claim of ITC, C1–C5 interest and late fee.

`match_heading()` compares on a *squashed* form — lowercase, every non-alphanumeric removed — so it absorbs the stray spaces `pdftotext` leaves inside words. `BOUNDARY` holds group headings such as "Excess claim of ITC" and "Late fee calculation", which end a section but get no sheet of their own.

`<project>/pipeline/descriptions.py` — 75 lines

Maps the rows of `item_desc.xlsx` to the 21 parameter ids and hands the model a plain-English description alongside the official title. A bare title is thin evidence for recognising an answer that never names the defect. The mapping is asserted on import, so a changed spreadsheet fails loudly instead of silently mislabelling every cell.

`<project>/pipeline/inventory2.py` — 72 lines, stage 1

Walks the case folders and records every PDF with its size, page count and **subfolder**. The subfolder is recorded rather than assumed, because it is only the conventional name when the folder follows the convention.

The notice file is chosen by size — the signed DRC-01 is around 1 MB against a 42 kB portal covering form, verified on all seven multi-PDF notice folders. The reply file is deliberately **not** chosen here: one case has a 289-page "Annexure A to G" that dwarfs the four-page reply beside it, and another has a two-character scan as its largest file, so size is the wrong test and the text does not exist yet.

`<project>/pipeline/text2.py` — 157 lines, stage 2

`pdftotext -layout` first, because the department's tables survive only while the column spacing is intact. A PDF that yields under `GST_MIN_CHARS_PAGE` characters a page, or whose text is mostly control characters, is re-read with `pdftoppm -r 300` + `tesseract`, and the result is cached because it is the slow step.

Two thresholds here were learned from failures:

- **600 characters a page, not 100.** A ten-page reply extracted at 251 characters a page and passed as native text, because its covering letter has a text layer and the reply letter behind it is a photograph. What reached the model was a list of annexure titles, and the case was recorded as "No reply" against a reply that was in the file all along. Across all 49 reply files in the main corpus the two mixed documents sit at 251 and 302 characters a page and the next file up is at 1206, so 600 falls inside a wide gap.
- **Compare letters, not length.** One PDF returned 39,592 characters of which four were letters. The garbled-text check fired and OCR ran — and the fallback kept the junk, because it was longer.

`<project>/pipeline/vlm_ocr.py` — 220 lines, stage 2b

Re-reads the scans with a vision model, because tesseract gets the words but destroys the tables, and a GST reply is mostly tables. 150 dpi PNG per page, six pages in flight.

tesseract's read is **kept**, beside the new file as `<name>.tess.txt`, and used as an independent witness. A generative model told to transcribe will invent rather than emit nothing: on one 26-page scan it reached a page it could not read and produced a calculus chapter followed by an NGO income statement, which then passed every downstream check — because by that point the invention *was* the source document. So each page is judged. A model answer far longer than tesseract's read of the same image, with almost no three-word runs in common, is discarded in favour of tesseract; a page the model filled with prose where tesseract saw nothing is dropped entirely. The prompt also instructs it to return `<<BLANK>>` for a page it cannot read.

If the model call fails outright, the page falls back to tesseract's text and the failure is recorded in the verdict — so an endpoint outage degrades the reading rather than losing the document.

`<project>/pipeline/scn_split.py` — 237 lines, stage 3, no model

Walks the notice line by line, opens a section on a parameter heading, closes it at the next heading or group heading, and **stops the document dead** at `Summary :` or "The total tax payable on account of these deficiencies". That cut drops 57% of all notice lines on the main corpus — the summary table, the "it is proposed to assess" paragraph and every annexure. The two notices with no Summary line fall back to a legibility heuristic.

`trade_name()` handles four different header layouts — letter style, label-then-value, value-then-label, and name-only-in-the-subject-line — and rejects labels and GSTINs. The first version returned a blank name for 41 of 42 cases.

Output is verbatim by construction: each cell is a slice of the cached text file, so nothing in the notice column can be paraphrased or invented.

`<project>/pipeline/notice_tables.py` — 225 lines, stage 3b

The department's tables are wider than the page, so a cell wraps and one figure arrives as two. This asks the model to rebuild the table, then **re-checks every figure against the raw text**: each must be a contiguous digit run in the source, or two runs joined — which is exactly what a wrapped cell is. A table that fails the check keeps its raw text rather than showing a repaired figure nobody verified.

`<project>/pipeline/reply_llm.py` — 470 lines, stage 4

The only stage that judges meaning. Keyword matching cannot work on this side: taxpayers reproduce a departmental heading when they feel like it and otherwise write "Query No: 2" or nothing at all.

It picks the arguing file by phrase density rather than size, caps it at 1,200 lines, windows it at 500 lines with 50 of overlap, and asks for the answer to each id **that case's own notice raised** — ids outside that list are rejected by the parser, not merely discouraged in the prompt.

Then it checks the answer. Every 4–12 digit figure must exist in the source text, and the wording must overlap it (five-word shingles, `GST_MIN_GROUND`, default 0.55). A cell that fails falls back to the verbatim slice the model pointed at; a cell with nothing to fall back on becomes `No reply`.

Five defences here exist because their absence caused silent damage:

- **12,000 output tokens.** At 4,000 the JSON was cut off mid-table, `json.loads` failed, and 15 of 48 documents silently produced zero items — every one read as "No reply".
- `salvage()` recovers complete objects from a truncated answer, and tries every closed object rather than only the outermost, since a cut inside the items array leaves the outer brace open — the one case the function exists for.
- **Trailing-comma tolerance.** A comma before a closing brace is invalid JSON and entirely normal model output; rejecting the response whole turned three correctly-read documents into 19 defects and 19 "No reply".

- **Single-case runs persist.** `reply_llm.py <GSTIN>` used to print its result and return, caching nothing, so the obvious repair — fix a document, re-run that case, rebuild — produced a workbook still built from the stale result.
- **"Copy, do not describe."** A larger model flattened the taxpayer's tables into prose: "Taxes payable as per SCN: CGST 84,90,528" where the reply printed those words as table rows. Every figure was right, so the figure check passed; the grounding check caught it and five of nineteen cells fell back. Saying so explicitly took those cells to 19 of 19 verified.

`<project>/pipeline/build2.py` — 222 lines, stage 5

21 sheets plus a Contents sheet. A case appears on a sheet only if the **notice** raised that parameter, so there is no row without a defect. Columns: GSTIN, trade name, notice defect, taxpayer reply, officer finding (blank). Text columns are Menlo 9pt so the `-layout` tables stay aligned. Repaired tables are used only where they were figure-checked. Control characters are stripped, because a single one aborts the whole save.

`<project>/pipeline/verify2.py` — 165 lines, stage 6

Fails loud. Each check exists because that class of mistake has already been made once:

- every notice cell is a literal substring of its cached source text
- nothing from below the Summary line reached a cell
- no row exists whose notice cell is empty
- only the 21 parameters appear
- every repaired table used was figure-checked
- every reply cell is verified model text, a verbatim fallback, or `No reply`
- no reply exists for a defect the notice never raised
- figures in a scanned-page reply were seen by tesseract too
- coverage floors for cases and cells, measured against **this dataset's own best run** at 90%, remembered in `coverage.json`. A first run has nothing to fall short of, and only a clean run raises the mark.

`<project>/pipeline/qwen.py` — 122 lines

The model client. One streamed completion, thinking off.

Streaming is not cosmetic: Cloudflare fronts the public endpoint and kills a connection that has sent nothing for about 100 seconds, so every request long enough to matter died with HTTP 524 and retrying hit the same wall. It also 403s the default Python user agent, hence the explicit header.

The key comes from `GST_API_KEY`, else `./.gst_api_key`, else `~/.gst_api_key` — never a command-line argument, so it stays out of shell history.

`strip_think()` removes `<think>` blocks from models that reason inline, including an unclosed one left by a truncated answer. The vision settings (`VLM_URL`, `VLM_MODEL`, `VLM_KEY`) default to the main ones, so a multimodal endpoint does both stages and only a text-only model needs them set separately.

`<project>/pipeline/demo_case.py` — 256 lines

The demo view. Runs the same stages against one folder and prints each defect with the taxpayer's answer beside it. It needs nothing prepared — no inventory, no caches, no dataset configuration — and writes no shared state, so a demo run cannot disturb a finished register. Accepts a path, a folder name or a GSTIN; repairs the notice tables concurrently; prints a 1,200-character preview per cell unless `--full` is given.

---

3. Data files

`<project>/item_desc.xlsx`

22 rows: the official description of each scrutiny parameter. Read by `descriptions.py` and fed to the model so it can recognise an answer that never names the defect. The only workbook committed to the repository, because the pipeline reads it.

`<project>/Parameters_TN.pdf`

The source document the 21 parameters were taken from. Not read at runtime — `params.py` holds the transcribed list.

---

4. Documentation

path	what it is
`<project>/run.md`	how to run it, for someone who has not used a terminal
`<project>/DOCUMENT.md`	this file
`<project>/DOCUMENT.pdf`	this file, printed
`<project>/README.md`	the short version: what it produces and the two commands
`<project>/CLAUDE.md`	the build plan and the measured facts behind each decision
`<project>/.gitignore`	keeps documents, caches, workbooks and credentials out of git

---

5. Generated directories

None of these are in git, so `git pull` never touches them.

path	what it holds	safe to delete?
`<project>/bin/micromamba`	the package manager, downloaded once	yes, after setup
`<project>/env/`	Python 3.12, poppler, tesseract, openpyxl	yes, rebuild with micromamba
`<project>/.conda/`	micromamba's package cache	yes, after setup
`<project>/.cache/`	pip, XDG, matplotlib and huggingface caches	yes
`<project>/tmp/`	OCR page renders during a run	yes, always
`<project>/work2/<dataset>/text/`	extracted and OCRed text, `.ocr` / `.vlm` flags, `.tess.txt` witnesses	yes, but OCR is then paid again
`<project>/work2/<dataset>/inventory.json`	every PDF with size, pages, subfolder	yes
`<project>/work2/<dataset>/scn.json`	the notice split, verbatim slices	yes
`<project>/work2/<dataset>/notice_fixed.json`	repaired notice tables	yes
`<project>/work2/<dataset>/reply_cache/`	one JSON per case — the resumable unit	yes, but the model runs again
`<project>/work2/<dataset>/reply.json`	merged replies, what the workbook is built from	yes
`<project>/work2/<dataset>/coverage.json`	best case and cell counts seen so far	yes, resets the floor

---

6. Datasets on this machine

Case documents are never committed — they are taxpayer records. Each dataset is a folder of case folders, and its workbook is written inside it.

path	what it is
`<project>/Test4_Notices/`	four cases from the main corpus, for testing the commands
`<project>/Test4_Notices/Test4_Notices.xlsx`	the workbook that folder produced
`<project>/Reply_WO_Parameter_Wise/`	three cases

A case folder holds the notice and the reply in separate subfolders:

```
<project>/Test4_Notices/33AAACV9956B2ZA_GSTR9/
├── DRC01_SCN/          the notice PDF
└── DRC 01 Reply/       the taxpayer's reply PDF
```

---

7. The browser tool

`<project>/web/` is a separate, unrelated tool: one static page that turns a single signed DRC-07 order into a three-column para-wise statement in the browser. Nothing in the batch pipeline uses it.

path	what it is
`<project>/web/index.html`	the whole application, one file
`<project>/web/pdf.min.js`, `<project>/web/pdf.worker.min.js`	pdf.js, for reading the PDF in the browser
`<project>/web/README.md`	its own documentation
`<project>/web/deploy/`	tmux and cloudflared scripts for serving it
`<project>/web/test/segregate.test.mjs`	asserts every rendered column is verbatim from the PDF
`<project>/web/test/tidy.test.mjs`	covers the table rebuild